

Module 23: Apex Testing

Apex Testing:

SDLC:

What is the purpose of testing?

1. We test the Apex code for each possible user input. and check that the code is working fine for each user input or not.

--> positive numbers

--> negative numbers

--> decimal numbers

--> it will give good quality of code.

2. Testing is required for deployment. to deploy the apex class, 75% of code should be tested. to deploy the Apex trigger, 1% of code should be tested.

23.1 Get Started with Apex Tests

23.2 Test Apex Triggers

23.3 Using seeAll Data

23.4 Using Test Setup methods

23.5 Using runAs method

23.6 Using startTest and stopTest

23.1 Get Started with Apex Tests

Apex Testing Definition

- Testing is an essential part of any software development lifecycle. It allows you to proactively identify and fix issues in your code before they can impact your users.
- Apex Testing is a unit testing framework for Apex code in Salesforce.
- It is the process of isolating and verifying the functionality of individual Apex classes and methods in Salesforce. It's the most granular type of Apex testing, focusing on the smallest building block of your code.
- It ensures code quality, uncover defects & prevents bugs, reliability, enhance confidence in deployments and achieve code coverage requirements.

Code Coverage requirement for Deployment

- Code coverage measures the percentage of code that is executed when the tests are run.
- Ensures that the code has been tested and works as expected.
- Identifies untested parts of the codebase.
- Salesforce requires a minimum of 75% code coverage for all Apex code.
- This is a mandatory requirement for deployment to production.
- Calls to System.debug are not counted as part of Apex code coverage in unit tests
- Code coverage is calculated based on the number of lines of code that are executed during tests.
- Formula:

$$\text{Code Coverage} = (\text{Number of Lines Executed} / \text{Total Number of Lines}) \times 100$$

Code Coverage requirement for Deployment

Tools to check Code Coverage:

- Developer Console:
Navigate to "Tests" tab to view code coverage.
- Salesforce Setup:
Setup > Apex Test Execution.
- Salesforce DX:
Command-line interface for detailed reports.

Test Class

- Test classes are declared with the `@isTest` annotation. This marks the class as containing unit tests.
- Can have public or private access specifier, but the default is private.
- Methods that test the code, annotated with `@isTest` or `testMethod(deprecated)`
- Test methods must be static and doesn't take any arguments.
- Test methods doesn't return any value (void).

Creating Test Data

- Salesforce records that are created in the test methods aren't committed to database.
- They're rolled back when the test execution finish.
- Rollback behavior is handy for testing because you don't have to clean your test data after the test execution.
- Creating Test Data can be done in some ways,
 - Within Test methods
 - Using `@TestSetup` method
 - Using `seeAllData = true` (to access data directly from Salesforce objects)

We need the 'Test data' to test our Apex classes. The test data is generally the salesforce object records.

WE can create the test data in below 3 ways:

1. Within the test method itself.
2. We can create a test Set up method separately. and we can use this test data at multiple places.
3. by using `seeAllData=true`, the actual records of salesforce object will be available as 'Test data'.

Assertion in Test class

System and Assert class contains methods to assert various conditions with test methods, such as whether two values are the same, a condition is true, or a variable is null.

Method	Description
<code>Assert.areEqual(expected, actual, message)</code>	Checks if two values are equal. Provide expected, actual values, and an optional message for assertion failure.
<code>Assert.areNotEqual(unexpected, actual, message)</code>	Checks if two values are not equal. Provide unexpected, actual values, and an optional message for assertion failure.
<code>Assert.isNull(value, message)</code>	Checks if a value is null. Provide the value to check and an optional message for assertion failure.
<code>Assert.isNotNull(value, message)</code>	Checks if a value is not null. Provide the value to check and an optional message for assertion failure.
<code>Assert.isTrue(expression, message)</code>	Checks if a boolean expression evaluates to true. Provide the expression and an optional message for assertion failure.
<code>Assert.isFalse(expression, message)</code>	Checks if a boolean expression evaluates to false. Provide the expression and an optional message for assertion failure.
<code>System.assertEquals(expectedValue, actualValue, message);</code>	Asserts that two values are equal. Provide expected, actual values, and an optional message for assertion failure.
<code>System.assertNotEquals(unexpectedValue, actualValue, message);</code>	Asserts that two values are not equal. Provide unexpected, actual values, and an optional message for assertion failure.
<code>System.assert(condition, message);</code>	Asserts that a condition is true. Provide the condition to check and an optional message for assertion failure.

Demo

Write a test class to test the functionality of “DMLOperations” class – “updateCandidateRecord” method by creating test data for all scenarios.

```
1 public class DmlOperations {
2     public static void updateCandidateRecord()
3     {
4         Candidate__c cd = [SELECT Years_of_Experience__c
5                             FROM Candidate__c
6                             WHERE Last_Name__c='Jane'
7                             LIMIT 1];
8         cd.Years_of_Experience__c=3;
9         update cd;
10    }
11 }
```

Demo

```
@isTest
private class DmlOperationsTest {
    static Map<Id,Candidate__c> candMap = new Map<Id,Candidate__c>();
    private static void testData(){
        Candidate__c cand1 = new Candidate__c(First_Name__c='Jane');
        Candidate__c cand2 = new Candidate__c(Last_Name__c='Jane');
        Candidate__c cand3 = new Candidate__c();
        List<Candidate__c> candList = new List<Candidate__c>{cand1,cand2,cand3};
        Database.insert(candList,false);
        candMap = new Map<Id,Candidate__c>(candList);
    }
    @isTest
    private static void testCandidates(){
        testData();
        DmlOperations.updateCandidateRecord();
        List<Candidate__c> candList = [SELECT Id FROM Candidate__c WHERE ID IN :candMap.keySet() AND Years_of_Experience__c=3];
        System.assertEquals(1,candList.size());
    }
}
```

TestRun @ 3:19:22 pm	0	1	Progress Report	0%	0/3
DmlOperationsTest	0	1	DmlOperations	100%	4/4
testCandidates			testCandidates	0%	0/0

Copyright © 2023 Accenture. All rights reserved. 13

23.2 Test Apex Triggers

Demo

Write a Test class to test the functionality of "PositionTrigger"- whenever a Position is created, update Status value to 'New'.

I

Solution:

@isTest

private class PositionTriggerHandler_Test {

 @isTest

 private static void testStatusUpdate(){

Position__c pos = new Position__c(Name = 'Developer', Job_Description__c =
 'Developer develops the code');

 insert pos;

Position__c posStatus = [SELECT Name, Status__c FROM Position__c WHERE Name =
 'Developer'];

 System.assertEquals('New',posStatus.Status__c);

 }

}

Copyright © 2023 Accenture. All rights reserved.

To-Do

Write a class to test the functionality of "OpportunityTrigger"- 'Closed won deals in Opportunity' cannot be deleted.

23.3 Using `seeAllData` Data

Purpose of `seeAllData` attribute

- The `seeAllData` attribute in Apex testing is used to determine whether a test class or test method has access to the organization's data.
- By default, test methods do not have access to data in your organization and can only access data that is created within the test itself.
- This is to ensure that tests are isolated and do not depend on the state of the organization's data, leading to more predictable and reliable tests.
- By default, test methods do not have access to the organization's data (`seeAllData=false`). This means that any data required for the test must be created within the test method or annotated with `@TestSetup`.
- Setting `seeAllData=true` allows the test method to access the existing data in the organization. This can be useful for specific scenarios where you need to validate behavior against existing data.

Syntax of `seeAllData` attribute

- Class Level

```
@isTest(seeAllData=true)
private class MyTestClass {
    // Test methods here have access to all data
}
```
- Method Level

```
@isTest
private class MyTestClass {
    @isTest(seeAllData=true)
    private void myTestMethod() {
        // Only this method has access to all data
    }
}
```

Demo

Write an apex class "Account Manager" to fetch Account Name for the provided Id, return NULL if Id not found in Account object. Test the apex class by fetching Id from database using `SeeAllData = true`.

```
1 public class AccountManager {
2     public static String getAccountName(String accountId) {
3         Account acc = [SELECT Name FROM Account WHERE Id = :accountId LIMIT 1];
4         return acc.Name;
5     }
6 }
```

Demo

```

1  @isTest
2  private class AccountManagerTest {
3      @isTest(SeeAllData=true)
4      private static void testGetAccountName() {
5          Account existingAccount = [SELECT Id, Name FROM Account LIMIT 1];
6          Assert.assertNotNull(existingAccount, 'An existing account should be found');
7          String accountName = AccountManager.getAccountName(existingAccount.Id);
8          Assert.areEqual(existingAccount.Name, accountName, 'The account name should match the expected value.');
```

✓	TestRun @ 4:48:46 am	0	2
✓	AccountManagerTest	0	2
✓	testGetAccountName	0:00	
✓	withoutSeeAllData	0:00	

➤

Copyright © 2023 Accenture. All rights reserved. 24

23.4 Using Test Setup methods

Definition

- The `@TestSetup` method is used to create common test data that can be used across multiple test methods within a test class.
- This method runs once for the test class and the test data it creates is rolled back after all tests in the class complete.
- If a class contains test setup method, before any test method in the class - the framework executes the test setup method first
- Eliminate repetitive data creation in each individual test method.
- This helps improve the efficiency and maintainability of test classes by avoiding redundant code for setting up test data.

Syntax

```
@isTest
private class MyTestClass {
    @testSetup
    private static void setupTestData() {
        // Create and insert test data that will be shared across test methods
    }
    @isTest
    private static void myTestMethod() {
        // Test logic that utilizes the shared test data
    }
}
```

Demo

Write a class to return Candidate records those who are Currently Employed. Write a test class to test the functionality by creating test data for all scenarios using Test Setup.

```
public class CandidateCurrentlyEmployed {
    public static List<Candidate__c> getCurrentlyEmployed(){
        return [SELECT Name, Currently_Employed__c FROM Candidate__c WHERE
        Currently_Employed__c=true];
    }
}
```

Demo

```
1  @isTest
2  private class CandidateCurrentlyEmployed_Test {
3      @testSetup
4      private static void candidateDataCreate(){
5          Candidate__c cand1 = new Candidate__c(First_Name__c='Candidate 1',Currently_Employed__c=true);
6          Candidate__c cand2 = new Candidate__c(First_Name__c='Candidate 2',Currently_Employed__c=false);
7          Candidate__c cand3 = new Candidate__c(First_Name__c='Candidate 3');
8          List<Candidate__c> candList = new List<Candidate__c>{cand1,cand2,cand3};
9          insert candList;
10     }
11     @isTest
12     private static void testEmployedCandidates(){
13         List<Candidate__c> candEmpList = CandidateCurrentlyEmployed.getCurrentlyEmployed();
14         System.assertEquals(1,candEmpList.size());
15     }
16     @isTest
17     private static void testUnEmployedCandidates(){
18         Integer totalCand = [select count() FROM Candidate__c];
19         List<Candidate__c> candEmpList = CandidateCurrentlyEmployed.getCurrentlyEmployed();
20         System.assertEquals(2,totalCand-candEmpList.size());
21     }
22 }
```


Definition

- The runAs method in Salesforce Apex is used to execute a block of code as a specific user.
- This is particularly useful for testing purposes, where you need to simulate the behavior of code when executed by users with different profiles, roles, or permissions.
- The runAs method is part of the System class and allows you to create more realistic test scenarios by mimicking the behavior of different users.

Syntax

@isTest

```
private class TestRunAs {
```

```
    @isTest
```

```
    private static void testRunAs() {
```

```
        Profile prof = [SELECT Id FROM Profile WHERE Name='Standard User'];
```

```
        User usr = new User(Alias = 'standt', Email='standarduser@testorg.com',
```

```
            EmailEncodingKey='UTF-8', LastName='Testing', LanguageLocaleKey='en_US',
```

```
            LocaleSidKey='en_US', ProfileId = prof.Id,
```

```
            TimeZoneSidKey='America/Los_Angeles',
```

```
            Username='standarduser' + DateTime.now().getTime() + '@testorg.com');
```

```
        System.runAs(usr) {
```

```
            // The following code runs as user 'usr'
```

```
        }
```

```
    }
```

```
}
```

Demo

Write a class to return the list of offer records based on the user's access and test the functionality - Interviewer profile has no access to Offer object. Test the functionality by run as user from Interviewer profile.

```
1 public with sharing class OfferAccess {
2     public static List<Offer__c> offerRecords(){
3         List<Offer__c> offList = [select Name from Offer__c];
4         return offList;
5     }
6 }
```

Demo

```
1 @isTest
2 * private class OfferAccess_Test {
3     @isTest
4     private static void testInterviewerProfileAccess(){
5         Offer__c offRec = new Offer__c();
6         offRec.Status__c = 'Sent';
7         offRec.Offer_Date__c = Date.newInstance(2024,09,10);
8         offRec.Offer_Expiration_Date__c = Date.newInstance(2024,09,17);
9         offRec.Actual_Salary__c = 40000;
10        insert offRec;
11        List<Offer__c> offSystemContext = OfferAccess.offerRecords();
12        Assert.areEqual(1,offSystemContext.size());
13        Profile prof = [SELECT Id FROM Profile WHERE Name='Interviewer'];
14        User usr = new User(Alias = 'standt', Email='standarduser@testorg.com', EmailEncodingKey='UTF-8', LastName='Testing', LanguageLocaleKey='en_US',
15                           LocaleSidKey='en_US', ProfileId = prof.Id, TimeZoneSidKey='America/Los_Angeles', Username='Interviewer' + DateTime.now().getTime() + '@testorg.com');
16        System.runAs(usr){
17            List<Offer__c> offUserContext = OfferAccess.offerRecords();
18            System.assertEquals(0,offUserContext.size());
19        }
20    }
21 }
```

23.6 Using startTest and stopTest

Definition

- In Salesforce Apex, the Test.startTest and Test.stopTest methods are used to manage the context of unit tests.
- These methods help ensure that tests are isolated and can make better use of governor limits.
- There should be only one startTest and stopTest within a method.
- They reset the governor limits between startTest and stopTest, providing a fresh set of limits for the actual test execution.
- They ensure that asynchronous operations like future methods, batch jobs, and queueable jobs are properly handled within the test.

Syntax

```
@isTest
private class myClass_Test {
    @isTest
    private static void myTestMethod() {
        // Create test data
        Test.startTest();
        // Actual apex code testing
        Test.stopTest();
    }
}
```

Example

```
1 @isTest
2 private class AccountUpdation_Test {
3     @isTest
4     private static void testAccountUpdate() {
5         Account acct = new Account(Name='Test Account');
6         insert acct;
7         acct.Description = 'Updated Account description';
8         update acct;
9         System.debug('DML Statement Limit before Start Test -->'+Limits.getDMLStatements());
10        Test.startTest();
11        System.debug('DML Statement Limit within Start Test and Stop Test -->'+Limits.getDMLStatements());
12        Test.stopTest();
13        System.debug('DML Statement Limit after Stop Test -->'+Limits.getDMLStatements());
14    }
15 }
```

Execution Log		
Timestamp	Event	Details
07:35:59:111	USER_DEBUG	[9][DEBUG]DML Statement Limit before Start Test -->2
07:35:59:112	USER_DEBUG	[11][DEBUG]DML Statement Limit within Start Test and Stop Test -->0
07:35:59:112	USER_DEBUG	[13][DEBUG]DML Statement Limit after Stop Test -->2

Demo

Write a Test Class to test the functionality of FutureMethodExample program.

```
public class FutureMethodExample {

    @future
    public static void updateCandidateCity(Set<Id> CandidateIds) {
        // Processing below code asynchronously
        List<Candidate__c> CandidatesToUpdate = [SELECT Id, Name, city__c FROM Candidate__c WHERE Id IN :CandidateIds];

        for(Candidate__c can : CandidatesToUpdate) {
            can.city__c = can.city__c + ' - Updated';
        }

        update CandidatesToUpdate;
    }
}
```

Demo

```
1 @isTest
2 private class FutureMethodExample_Test {
3     @testSetup
4     private static void testCandidate(){
5         Candidate__c cand = new Candidate__c(First_Name__c = 'Eric',City__c='Boston');
6         insert cand;
7         FutureMethodExample.updateCandidateCity(new Set<Id>{cand.Id});
8     }
9     @isTest
10    private static void testCandidateUpdate(){
11        Test.startTest();
12        Candidate__c candUpdated = [select City__c from Candidate__c];
13        Assert.areEqual('Boston - Updated',candUpdated.City__c);
14        Test.stopTest();
15    }
16 }
```

Apex Testing Best Practices

- One Test class per class
- Ensure Test data is created properly
- Aim for high code coverage
- Test for Positive, Negative and Exception Scenarios
- Test all Business logic
- Use Test setup methods
- Use Factory classes to setup test data
- Use Test.startTest and Test.stopTest
- Minimize usage of seeAllData=True
- Usage of Assert methods in each test methods